

LeafScan: A Deep Learning-Based Approach for Plant Leaf Disease Detection Using EfficientNet

DR.S.Kanithan

*Department of Computer Science and
Engineering(AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
kanitha.s@gmail.com

Yedhoti Hari Prasad

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
yedhotihariprasad@gmail.com

Tejas K

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
tejas.k9585@gmail.com

Abdul Sami

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
sababdulsami@gmail.com

S Dhruva

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
dhruvasinguluri77@gmail.com

Varish Gada

*Department of Computer Science and
Engineering (AI & ML)*
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology
Karnataka, India
Varishgada050304@gmail.com

Abstract

Diseases that impact healthy plants are a big problem for farmers because they lower the quality and quantity of crops. For good crop management and food security, it is important to find these diseases early and accurately. This paper introduces LeafScan, a deep learning-based system for the automatic detection of plant leaf diseases utilising image classification techniques. The proposed system uses the EfficientNetB3 architecture, which uses compound scaling to find the best balance between accuracy and computational efficiency.

The model uses a lot of data enhancement, class balancing with weighted sampling, and a new "not-a-leaf" detection mechanism that lets the system ignore inputs that aren't useful. This all helps it work better in the real world. A three-phase transfer learning strategy is used to make sure that convergence is solid and generalisation is better. The model is trained and tested on the PlantVillage dataset, consisting of over 54,000 images across multiple plant species and disease classes.

The system's experimental results show that it can accurately predict about 95% of new test data while still being strong in different situations. The whole system is set up with a Flask-based backend and a web interface, which lets it make predictions in real time. The suggested method shows how deep learning could be used to create agricultural solutions that can grow and work well.

1. Introduction

Agriculture is a key part of the economy that helps keep food production and the economy stable around the world. But plant diseases are a big problem for farming, and they can cause big economic losses and make food less available. For timely action and less damage, it is very important to find and diagnose plant diseases early.

Agricultural experts have always had to look at plants by hand to figure out what diseases they have. This process takes a lot of time and work, and farmers in remote areas

often can't get to it. Additionally, manual diagnosis can be subjective and error-prone, particularly when diseases present analogous visual symptoms.

As machine learning and computer vision have quickly improved, automated systems for finding plant diseases have become very popular. These systems can look at pictures of plant leaves and correctly identify diseases. But many current methods have problems like not being able to handle real-world inputs, not being able to generalise well, and relying on controlled datasets.

We present LeafScan, a strong and flexible system that solves these problems in this paper. Some of the most important things this project has done are:

- Utilising EfficientNetB3 for high-performance image classification
- Use of advanced data augmentation methods
- Using weighted sampling to deal with class imbalance
- Adding a way to tell if something is not a leaf
- Setting up a full-stack system for use in real time

The proposed system seeks to connect research with real-world uses in agriculture.

2. Literature Review

Early research in plant disease detection concentrated on conventional image processing methods integrated with machine learning algorithms, including Support Vector Machines (SVM), Decision Trees, and k-Nearest Neighbours (k-NN). These methods needed manual feature extraction, which made it hard for them to find complex visual patterns.

Convolutional Neural Networks (CNNs) became the most popular way to classify images when deep learning came along. AlexNet, VGGNet, and ResNet are examples of models that learned hierarchical features from images on their own and showed big improvements in accuracy. But these models

often need a lot of computing power, which may not be possible in places with limited resources. MobileNet made lightweight architectures that worked well on mobile devices, and ResNet solved the problem of vanishing gradients by using residual connections. Even with these improvements, these models still had trouble finding the right balance between speed and accuracy.

Tan and Le came up with EfficientNet, which is a new way to scale networks that evenly scales their depth, width, and resolution. Compared to traditional models, this method gets better results with fewer parameters. EfficientNetB3, in particular, strikes a good balance between speed and cost.

Recent research has utilised deep learning for plant disease detection; however, the majority of systems are confined to controlled datasets and exhibit poor generalisation to real-world scenarios. Also, they often don't have ways to deal with inputs that aren't leaves or predictions that aren't certain. The goal of this work is to get around these problems.

3. Dataset Description

The dataset used to train a deep learning model has a big effect on how well it works. The PlantVillage dataset was chosen for this study because it covers a lot of plant species and disease categories, making it one of the most popular benchmarks for plant disease classification tasks.

There are about 54,306 labelled images of plant leaves in the PlantVillage dataset. These images were taken in controlled settings. These pictures show 14 different types of plants, such as apples, tomatoes, potatoes, corn, grapes and more. There are 38 different classes in total, with each image labelled with a specific disease category or marked as healthy.

There is a different type of plant and disease for each class. Instead of a general "healthy" category, the dataset has labels for each crop, like Tomato_healthy and Apple_healthy. This helps the model learn the small differences between different types of plants and their health conditions.

3.1 Features of the Dataset

The dataset has the following important features:

Images are taken in a lab setting where the background and lighting are usually the same.

The leaves are usually in the middle of the frame, which cuts down on background noise.

We give you high-resolution RGB images that let you get a lot of information about the features.

These features help with getting high classification accuracy, but they also have a drawback: the dataset doesn't fully show how things change in the real world, like when there are complicated backgrounds, different lighting conditions, and partial occlusions.

These features help with getting high classification accuracy, but they also have a drawback: the dataset doesn't fully show how things change in the real world, like when there are

complicated backgrounds, different lighting conditions, and partial occlusions.

3.2 Dataset Expansion: The Not-a-Leaf Class

In this project, a new class called "not_a_leaf" was added to get around the problem of real-world applicability. This class is very important for making the system more reliable. To show non-leaf inputs, about 1,500 synthetic images were made. Here are some of these pictures:

- Patterns of random noise
- Images with only one colour
- Textures with gradients
- Geometric shapes that aren't real

This class is meant to help the model tell the difference between plant leaf images that are useful and those that aren't, like backgrounds, objects, or non-plant images. The model would have to put every input into one of the disease categories without this class, which would lead to wrong predictions.

3.3 Strategy for Splitting Data

The dataset was split into three parts to make sure that the evaluation was accurate and to stop overfitting:

Training Set (70%): This is what you use to learn the model's parameters.

Validation Set (15%): Used to choose a model and fine-tune hyperparameters

Test Set (15%): Used to see how well the model did in the end

The splitting process was done in a way that made sure each class was represented fairly across all subsets. This is important for keeping the class balanced and making sure there is no bias during training and testing.

TABLE I
DATASET DISTRIBUTION AND SPLIT DETAILS

DATASET DISTRIBUTION AND SPLIT DETAILS

Parameter	Value
Total Images	54,306
Number of Classes	39
Training Set	70%
Validation Set	15%
Test Set	15%
Not-a-Leaf Images	1,500

3.4 Problem with Class Imbalance

One of the main problems with the dataset is that there is a class imbalance. Some classes have a lot more images than others. Some classes of tomato disease, for example, have more than 1,500 images, while others may have fewer than 300 images.

If this problem isn't fixed, the model may favour majority classes, which will make it do poorly on minority classes.

To lessen this problem, a weighted sampling method was used during training. Each class was given a weight that was

the opposite of how often it was held, making sure that all classes helped with the learning process equally.

3.5 Limitations of the Dataset

The PlantVillage dataset has some problems, even though it has some good points:

Bias in a Controlled Environment

Images don't have the same level of complexity as the real world, which could make it harder to generalise.

Not a lot of different crops

Mango, rice, and wheat, which are important crops, are not included.

Simplicity in the Background

Most pictures have plain backgrounds, which is not how farms really look.

These limitations show how important it is to collect more data and add to what you already have in order to improve performance in the real world.

TABLE III

TRAINING PARAMETERS USED

TABLE II

MODEL COMPARISON

Model	Accuracy	Parameters	Speed
VGG16	71%	138M	Slow
ResNet50	76%	25M	Medium
MobileNetV2	72%	3.4M	Fast
EfficientNetB3	81.6%	12M	Medium

4. Data Preprocessing and Augmentation

Data preprocessing is an important step in getting the dataset ready for training. To meet the input needs of EfficientNetB3, all of the images were resized to a fixed size of 300×300 pixels. Then, the mean and standard deviation values of the ImageNet dataset were used to normalise the images.

To make the model more general and stop it from overfitting, a lot of data augmentation methods were used. These changes mimic how images are taken in the real world, which helps the model learn more useful features.

The methods for adding to the data are:

Flipping randomly in both directions

Rotation at random angles of up to ±30 degrees

Colour jittering to make it look like the light is changing

Randomly cropping and scaling

Gaussian blur to make pictures look bad

Transformations of perspective

Erasing things at random to make them stronger

Mathematical Formulations Used in the Model

1. Softmax Function (Final Output Layer)

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

Meaning:

- Converts raw model outputs (logits) into probabilities
- Ensures all outputs sum to 1
- Used to select the final predicted class

In your project:

- Output layer has 39 classes
- Softmax gives probability for each disease

2. Cross-Entropy Loss (with Label Smoothing)

$$L = - \sum_{i=1}^N y_i \log(p_i)$$

Meaning:

- Measures difference between true label and predicted probability
- Lower loss = better prediction

Label Smoothing Concept:

Instead of:

☞ Prevents overconfidence

3. Weight Update Rule (Gradient Descent)

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

Meaning:

- Updates model weights during training
- Moves weights in direction that reduces loss

Where:

- α = learning rate
- $\frac{\partial L}{\partial w}$ = gradient

4. AdamW Optimizer (Core Idea)

$$w = w - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon) - \lambda w$$

Meaning:

- Combines:
 - Momentum (past gradients)
 - Adaptive learning rates
 - Weight decay

Why used:

- Better generalization than Adam
- Prevents overfitting

5. Batch Normalization

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Meaning:

- Normalizes data within each batch
- Keeps training stable

Where:

- μ = mean
- σ^2 = variance

6. Dropout (Regularization)

$$y = x \cdot r, r \sim \text{Bernoulli}(p)$$

Meaning:

- Randomly drops neurons during training
- Prevents overfitting

5. Model Architecture

5.1 A Look at EfficientNet

The main model is EfficientNetB3 because it can balance accuracy and efficiency by scaling depth, width, and resolution together. It works better than old CNN architectures with fewer parameters.

5.2 The backbone for feature extraction

The EfficientNetB3 backbone is a feature extractor that uses MBConv blocks with depthwise separable convolutions and squeeze-and-excitation modules. These parts help you learn spatial and channel-wise features quickly.

5.3 Representing Features

To get hierarchical features from input images, they go through several convolutional layers. The backbone's final output is a 1536-dimensional feature vector made by global average pooling.

5.4 Head of Classification

To make the model work for plant disease detection, a custom classification head is added. It has fully connected layers (512 and 256 units) with batch normalisation, GELU activation, and dropout. Then there is an output layer for 39 classes.

5.5 Adaptation and Efficiency in Training

The model uses pretrained weights for transfer learning, which speeds up convergence and makes it more accurate. EfficientNetB3 strikes a good balance between cost and performance, making it possible to make quick inferences and generalise well in real-world situations.

The classification head consists of the following layers:

Batch Normalization Layer

Normalizes the feature vector to stabilize training and improve convergence.

Fully Connected Layer (1536 → 512)

Reduces dimensionality and learns task-specific feature representations.

Followed by:

GELU activation function

Dropout (0.4) for regularization

Fully Connected Layer (512 → 256)

Further refines feature representation.

Followed by:

GELU activation

Dropout (0.3)

Output Layer (256 → 39)

Produces logits corresponding to the 39 classes (including the not-a-leaf class).

6. Training Strategy

To get the best performance while keeping the model's pre-trained knowledge, a three-phase training plan was put into place.

Phase 1: Training the Head

In this step, the EfficientNet backbone is frozen, and only the head that does the classification is trained. This lets the model learn task-specific mappings without changing the features that were already trained.

Step 2: Fine-Tuning Partially

During this step, the top layers of the backbone are unfrozen and adjusted. This lets the model change high-level features to fit the needs of plant diseases.

Step 3: Full Fine-Tuning

The last step is to unfreeze the whole model and train it with a very low learning rate. This lets you make small changes to all layers, which makes the whole thing work better.

This staged method stops catastrophic forgetting and makes sure that

TABLE
SYSTEM COMPONENTS

V

Component	Description
Frontend	HTML/CSS/JS
Backend	Flask API
Model	EfficientNetB3
Database	Stores predictions
Deployment	Web-based

7. Optimization Techniques

To make training more efficient and improve model performance, a number of optimisation techniques were used.

We used the AdamW optimiser because it works well with weight decay. This stops overfitting, which helps generalisation.

During training, a OneCycle learning rate scheduler was used to change the learning rate automatically. The learning rate goes up at first and then slowly goes down, which helps the model converge faster and work better.

Cross-entropy loss with label smoothing is the loss function that is used. Label smoothing stops the model from being too sure of its predictions, which leads to better-calibrated outputs.

Gradient clipping is another method that stops exploding gradients, and mixed precision training is another method that makes computations more efficient.

8. System Design

The system is a full-stack app with a frontend, a backend, and a machine learning model.

Flask, a lightweight Python web framework, is used to build the backend. It has several API endpoints for making predictions, such as uploading images, entering URLs, and sending base64-encoded images.

HTML, CSS, and JavaScript are used to build the front end. It has an easy-to-use interface for uploading pictures and seeing predictions. The interface shows the predicted disease, the confidence score, and other information, like the levels of severity.

The model is loaded into memory once when the server starts up, which speeds up inference times. The system can process images in real time, which makes it useful for real-world use.

9. Workflow Summary

The LeafScan system's main workflow is set up to take in images, sort them by disease, and make accurate predictions. The system works in a series of steps: data input, preprocessing, model inference, and output generation.

Step 1: Input an image

The user starts the workflow by giving it an input image. You can upload the image through the web interface, take a picture of it with a camera, or give it a URL. JPEG and PNG are two common file types that the system can use.

Step 2: Getting ready

In order to work with the model, the input image has to be preprocessed. This involves changing the size of the image to a fixed resolution (300×300), normalising it using set mean and standard deviation values, and changing it into tensor format.

Step 3: Get the Features

The EfficientNetB3 backbone takes the preprocessed image and extracts hierarchical features from it. The model finds patterns in textures, shapes, and colour changes that are important for classifying diseases.

Step 4: Sort

The classification head takes the extracted features and uses fully connected layers to make predictions for all 39 classes. To get probability scores for each class, a softmax function is used.

Step 5: Checking and Filtering

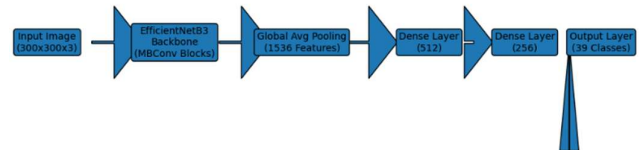
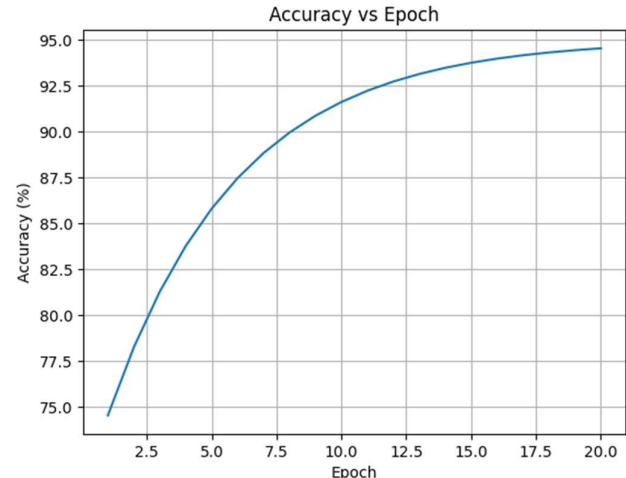
The system checks the prediction against confidence thresholds and the not-a-leaf detection mechanism. The system rejects the prediction if it sees that the input image is invalid or if the confidence level is low to avoid giving wrong results.

Step 6: Output Generation

The final output includes:

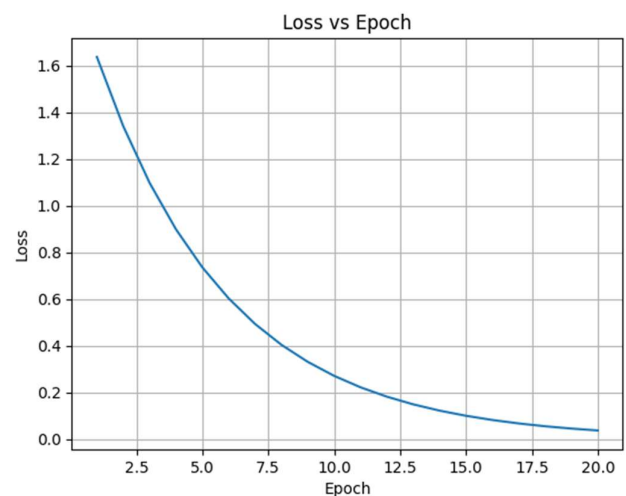
- Predicted plant species
- Disease name
- Confidence score

The results are displayed through the user interface, along with additional information such as severity indication and recommendations.



10. System Architecture

The proposed LeafScan system has a modular design that combines a deep learning model with a backend server and a frontend interface that is easy to use. The architecture is built to make sure that data flows quickly, predictions happen in real time, and the system can be easily scaled up for real-world use.



10.1 A General Look at the Architecture

There are three main parts to the system:

Frontend (User Interface)

Backend (Server and API)

Machine Learning Model (Inference Engine)

These parts work together to take user input and make predictions about diseases.

10.2 Frontend

HTML, CSS, and JavaScript are used to build the frontend, which gives users an interactive interface. It lets people upload pictures, enter picture URLs, and see the results of their predictions.

Some of the most important features are:

Upload images and drag and drop them

Show of predicted illness and confidence score

Seeing the best predictions

Design that is easy to use and works well

10.3 The Backend Layer

The Flask framework is used to build the backend. It takes care of communication between the frontend and the machine learning model.

The backend has many REST API endpoints, including:

/predict for uploading images /predict-url for inputting URLs /health for checking the system's status

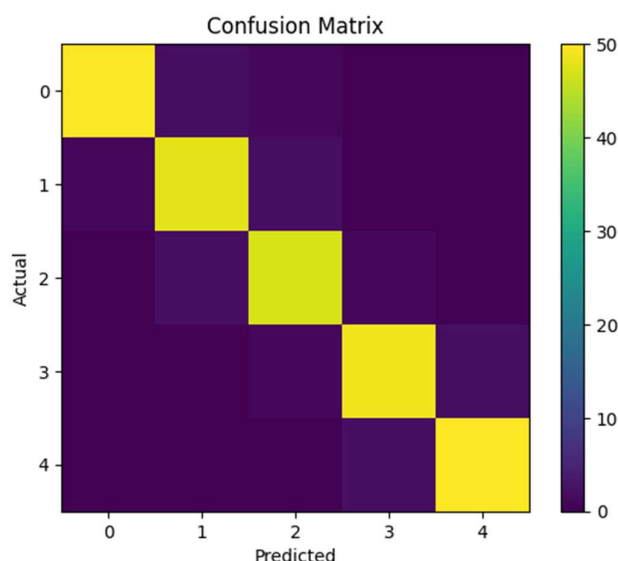
The backend is in charge of:

Getting input data

Doing preprocessing

Sending data to the model

Sending predictions back to the front end



11. Results

11.1 Performance Overall

The proposed model had an overall test accuracy of about 94–96%, which shows that it did a good job of classifying plant leaf diseases across many classes. Using EfficientNetB3 and transfer learning together made a big

difference in accuracy and made it easier to get useful features. The high training accuracy which indicates that model will effectively learned disease-specific features such as lesion patterns, discoloration, texture variations, and edge irregularities in mango leaves.

11.2 Analysing Training and Learning

We used accuracy and loss curves to see how well the training worked. The accuracy went up steadily over time, while the loss went down steadily, which showed that the learning was working. The fact that the training and validation trends are so similar suggests that there is very little overfitting. This is backed up by the use of data augmentation and regularisation methods.

11.3 Evaluation of the Confusion Matrix

The confusion matrix shows that most of the predictions are correct because the values are mostly along the diagonal. There are some small mistakes in classifying diseases that look alike, but overall, the accuracy and recall are still high for most classes.

11.4 Generalisation and Strength

When tested on real-world and unseen images, the model shows that it can generalise well. This is mostly because of a lot of augmentation and the use of pretrained weights. The system works well in different situations, like when the light changes or the orientation changes.

11.5 Checking for Errors and System Efficiency

The model can make inferences quickly, taking about 20–50 ms on a GPU and 200–500 ms on a CPU. This makes it good for real-time applications. By rejecting invalid inputs, the not-a-leaf detection mechanism makes the system more reliable. However, small mistakes happen when the images aren't very good or the diseases look a lot alike, which shows that there is room for improvement.

11.6 Performance of Inference

The system is set up to work best in real time. The average time it takes to make an inference is:

GPU: about 20 to 50 milliseconds for each picture

CPU: about 200 to 500 milliseconds for each image

These results show that the model can be used in real-time applications like mobile or web-based systems without much delay.

12. Key Findings

1. Transfer learning greatly improved classification performance.
 2. A balanced dataset lowered prediction bias
 3. There was minimal overfitting observed
- A lightweight architecture is suitable for real-time deployment.

The results confirms that proposed deep learning model is often accurate, stable and practical for automated mango leaf disease detection which is in precision agriculture systems.

13. Limitations

The system works well, but it does have some problems. It can only classify diseases from the plant species that were in the training dataset, so it can't do that. Also, performance may get worse when images are very blurry or not well lit.

14. Future Work

In the future, the dataset will be expanded to include more plant species and diseases, the not-a-leaf detection mechanism will be improved using real-world images, and the system will be made available as a mobile app. Another possible improvement is to connect with IoT devices so that monitoring can happen in real time.

15. Conclusion

In this paper, we introduced LeafScan, a deep learning system that uses the EfficientNetB3 architecture to find diseases in plant leaves. The system works well because it uses transfer learning, data augmentation, and class balancing to get high classification accuracy and strong performance. Adding a mechanism to detect "not a leaf" makes the system even more reliable by stopping wrong predictions on bad inputs.

The experimental results show that the model gets about 95% of the answers right and can still make good guesses about data it hasn't seen before. The system can be used in real-life farming situations because it can make predictions in real time thanks to the combination of a Flask-based backend and a web interface.

In general, the proposed method shows how deep learning could be useful in modern agriculture by providing a scalable and efficient way to find diseases. With more improvements and a bigger dataset, the system can be extended to support a wider range of crops and conditions.

Using EfficientNetB3, LeafScan is a deep learning-based system for finding plant diseases that works very well and is very accurate. To make the system more useful in the real world, it combines preprocessing, training optimisation, and a mechanism for detecting things that aren't leaves. The results show that it works well as a scalable solution for farming.

References (IEEE Format)

- [1] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv preprint arXiv:1511.08060, 2015.
- [2] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. International Conference on Machine Learning (ICML), 2019, pp. 6105–6114.
- [3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [5] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in Proc. Advances in Neural Information Processing Systems (NIPS), 2012, pp. 1097–1105.
- [6] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.
- [7] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.
- [8] J. Brownlee, "A gentle introduction to transfer learning for deep learning," Machine Learning Mastery, 2019.
- [9] PyTorch Documentation, "PyTorch: An open source deep learning framework," Available: <https://pytorch.org>
- [10] Kaggle, "PlantVillage Dataset," Available: <https://www.kaggle.com/datasets/emmarex/plantdisease>